



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

VLSI DESIGN PROJECT REPORT

Implementation of Digital Lock using FSM on FPGA Spartan-7



Group Members:

Aditya Sthavarmath (23070123008)

Adrish Purkayastha (23070123011)

Akshit Mathur (23070123014)

Kautik Verma (23070123073)



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Abstract:

Digital lock system requires more secure, predetermined and highly reliable hardware implementations to make sure about the data and physical security. This project directs the need to strong, real-time sequential code verification mechanism through design and Verilog implementation of four-digit digital combination lock based on synchronous Finite State Machines (FSM) structure to handle the sequential input states and give unlock and failed status flags.

This system integrates two hardware modules, a hardware debounce to filtering the mechanical switch noise for stability and guaranteeing the accurate clock cycle and input detection. Multiplexed seven-segment display driver which gives clear displaying the feedback of system status. The implemented system is simulated and synthesis in standard VLSI design flow confirming the correct state transitions, verifying four key sequence and strong noise immunity.

This project confirms the efficiency and flexibility of FSM based control logic for security and real time applications for FPGA, faulty tolerant solutions, making it reliable for embedded systems and automation systems.

1. Introduction:

1.1 Background and Motivation

In application of today's engineering, particularly where fast and reliable access control is needed, the speed and reliability of the base digital logic are of the highest concern. Traditional electronic lock and security door systems are generally based on mechanical input devices, which have the unfortunate character of signal degradation such as a random operation due to switch bounce and asynchronous timing of inputs. In the high assurance applications, the control logic must be implemented in dedicated hardware, such as in a Very Large-Scale Integration (VLSI) circuit, in order to provide deterministic functioning, to achieve high performance, and to be immune to software exploits. In the case of sequential security protocols, where the input sequence becomes important (i.e. combination locks), memory and control become stateful and this design is best achieved by a Finite State Machine (FSM) architecture.

1.2 Problem Statement

Central to the solution of this project is the design and implementation of a digital lock system that has high security and predictable operation on a hardware system. In particular, the system must be able to process a given, multi-step sequential input code (a four-digit combination) from noisy, asynchronous mechanical keys correctly. A solution that's sound and durable needs to have several properties: it must filter out any and all input noise, make its sequence validation



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

logic foolproof and give clear, immediate and power-effective visible feedback to the user. A failure to meet these hardware and sequential logic specifications results in an unsafe system that can be easily locked out or hacked.

1.3 Project Aims

The overall objective of this project consists in modelling and designing, and in proving a complete digital locking system using Verilog Hardware Description Language (HDL) aimed at synthesizing it onto FPGA. This objective then gets broken down into the following overall targets: To create a synchronous Moore-type FSM that will track four distinct input states in confirming a secret combination sequence. To produce a specialized debouncing module that will convert asynchronous mechanical key press to a single clean clock synchronized pulse that allows transition of states. Integrated seven segment multiplexed driver to dynamically non-flickeringly display system status (e.g. UNLC to indicate unlocked, LOC to indicate failed attempt). To demonstrate the effectiveness and accuracy of the system by successfully modelling and integrating the full architecture to illustrate its maturity, in order to carry out the physical implementations.

2. Theory and Design: Theoretical Design Principles

2.1 Design of Finite State Machine (FSM)

The main rationale for successive password validation is represented in a synchronous FSM. A synchronous design was selected in order to impose that the state changes occur only on rising edge of system clock, which will guarantee deterministic behaviour and will make timing analysis easier.

2.1.1 State Definition

The FSM is a Moore Machine, in which the outputs (unlocked and failed) are dependent only upon the current state. Six different states are required to keep track of the four-key combination, failure, and being unlocked until reset.

State Name	Binary Encoding	Function	Exit Condition(s)
IDLE	3'b000	Waiting for the first key press.	Any debounced key press moves to GOT_1.
GOT_1	3'b001	Acknowledged the first key press.	Next debounced press moves to GOT_2.
GOT_2	3'b010	Acknowledged the second key press.	Next debounced press moves to GOT_3.
GOT_3	3'b011	Acknowledged the third key press.	Next debounced press moves to UNLOCKED or FAILED.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

UNLOCKED	3'b100	Correct four-key sequence entered.	Remains here until reset is asserted.
FAILED	3'b101	Incorrect key entered at any step.	Remains here until reset is asserted.

2.1.2 State Transition Logic

The FSM uses a correctness `correct_so_far` flag which is set to true when reset. This flag is checked during key sequence: If the key does not match with the pre-determined sequence, the flag is immediately de-asserted. This design makes sure that the user is forced to go through all four input steps even if the failure occurs at one of the previous steps and the FAILED state is set. The final decision is only taken when the final, fourth key is pressed and this considers the fourth input as well as the `correct_so_far` flag.

3. System Structure and Implementation.

This section will describe the hierarchy of the digital lock system, explain the implementation methodology defined through Hardware Description Language (HDL) as well as the functional role of each of the instantiated modules.

In this section, the general system architecture is given.

The digital lock system has a modular hierarchical organization that is defined by four different Verilog modules, as shown in the block diagram below. With this method, there is certainty of clarity, reusability, and ease of independent testing on a subsystem.

The system uses the `top_module` to serve as the main wrapper in the system that would take care of the connectivity between core logic, input conditioning and the output display driver.

Module Name	Purpose	Key Inputs	Key Outputs
debounce	Filters electrical noise from mechanical buttons and outputs a single, clean pulse per press.	clk, reset, keys_in (4-bit raw input)	debounced_press
digital_lock	Implements the six-state FSM for sequential code validation.	clk, reset, debounced_press, keys_in (for comparison)	unlocked, failed



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

seven_segment_driver	Manages the time-division multiplexing and decoding to display status.	clk, reset, char1 to char4 (Status Codes)	segments_out, anodes_out
top_module	Instantiates and connects all sub-modules, handles status-to-display routing.	clk, reset_sw, keys_in	segments_out, anodes_out

3.3 Methodology of Implementation.

The whole design was to be implemented in Verilog HDL and synthesized to a particular FPGA platform (e.g. the Xilinx Spartan-7 device).

Behavioural Modelling: Modules code was first written as (digital lock) (debounce) (seven segment driver) according to the theoretical models designed in Section 2.

3.3.1 Digital Lock FSM

Digital_lock module also uses two distinct always blocks to execute the next-state and output logic, one which is configured to update the state synchronously (invoking posegde clk) and another which is configured to result in next-state and output generation which is always on (invoking always @(*)). The order is coded in as an active-low input: KEY1 → KEY0 → KEY2 → KEY3 and the state machine appropriately makes use of the correct_so_far flag to arrange failure reporting.

3.3.2 Debounce Circuit

A 21-bit counter is used in the debounce module, but it is set such that the number counter would reach the target of 20 ms in order to arrive at the debounce period. Most importantly, the raw button inputs (buttons_in) are filtered to pass through two sequential synchronizing flip-flops to reduce metastability before they are fed to the debouncing counter to provide flawless passage through the asynchronous input domain to the synchronous clock domain.

3.3.3 Seven-Segment Driver

A counter of 18 bits is used by the driver to produce the needed refresh frequency. The two upper bits of this counter are read out to cycle the four anodes in the same way as to produce a flicker free decline rate of about 122 Hz. The logic that enables character decoding takes care of the necessary segments that are active low that must be used to make a common anode display to display U, n, L, c, o, and a blank character.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

3.4 Working Principle:

- 1) Initialization: The system switched on or the reset switch pressed, the system enters in IDLE state. The failed and unlocked outputs are off and the seven-segment display will have a blank screen or a standby introductory message. The FSM correct so far flag (correct_so_far) is set to true and it is identified the first time it receives an input.
- 2) Input: The keys in signal gets to debounce module This is because this module is a 20 ms counter that is used to keep the key steady. It is not until this time that the push button produces a one-clock-cycle pulse on the debounced press signal which is technically clean. All the state changes are triggered by this pulse.
- 3) Verifying the sequence of FSM: State (GOT1 to GOT3): Once a successful trigger has been seen, the FSM compares the keys in that is currently typed in with the hoped-for passkey, in this step. When identifies the right key then the FSM will transition to the next sequential state (e.g. IDLE to GOT1). In case the key is wrong, the FSM will continue on to the next state but the correct so far flag will be set at once and forever until the reset.
- 4) Final Check (GOT3 to Final State): In the fourth and final key press, the FSM does a check, or rather, another check, that is, it not only looks at the value of a key in a sequence, but also looks at the value of the correct so far flag.
Success: In case of the correctness of the last key and the correct so far, the FSM goes to the UNLOCKED state, indicating that it is unlocked.
Failure: In case the end key was wrong or correct so far is wrong, the FSM goes to the FAILED state, causing the failed signal to be turned on.
- 5) Output Display (The Driver): The failed and unlocked signals are followed in the top module. When unlocked high, display code will indicate U n L c. In case of high failed, display code of L o c. To visually display the status message, the seven segment driver rides through the right segments and anodes with time-division multiplexing.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Code:

```
module top_module(  
    input clk,  
    input reset_sw,  
    input [3:0] keys_in,  
    output [6:0] segments_out, // Reverted to 7-bit  
    output [3:0] anodes_out  
);  
    // Internal signals  
    wire unlocked_status;  
    wire failed_status;  
    // Instantiate the digital lock FSM  
    digital_lock uut_lock (  
        .clk(clk),  
        .reset(reset_sw),  
        .keys_in(keys_in),  
        .unlocked(unlocked_status),  
        .failed(failed_status)  
    );  
    // Logic to select which characters to display  
    reg [3:0] d1, d2, d3, d4;  
    // Character codes for the 7-segment driver  
    localparam CHAR_U = 4'h0;  
    localparam CHAR_N = 4'h1;  
    localparam CHAR_L = 4'h2;  
    localparam CHAR_C = 4'h3;  
    localparam CHAR_O = 4'h4;  
    localparam BLANK = 4'hF;
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

always @(*)

begin

if (unlocked_status) begin

// Display "unlc"

d1 = CHAR_U;

d2 = CHAR_N;

d3 = CHAR_L;

d4 = CHAR_C;

end

else if (failed_status) begin

// Display " loc"

d1 = BLANK;

d2 = CHAR_L;

d3 = CHAR_O;

d4 = CHAR_C;

end

else begin

// Display is blank otherwise

d1 = BLANK;

d2 = BLANK;

d3 = BLANK;

d4 = BLANK;

end

end

// Instantiate the 7-segment display driver

seven_segment_driver uut_display (

.clk(clk),

.reset(reset_sw),

.char1(d1),



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
.char2(d2),  
.char3(d3),  
.char4(d4),  
.segments(segments_out),  
.anodes(anodes_out)  
);  
endmodule
```

```
module digital_lock(  
    input clk,  
    input reset,  
    input [3:0] keys_in,  
    output reg unlocked,  
    output reg failed  
);  
    // FSM States  
    localparam IDLE = 3'b000;  
    localparam GOT_1 = 3'b001;  
    localparam GOT_2 = 3'b010;  
    localparam GOT_3 = 3'b011;  
    localparam UNLOCKED = 3'b100;  
    localparam FAILED = 3'b101;  
    // State registers  
    reg [2:0] current_state, next_state;  
    // This flag tracks if the sequence is correct up to the current point.  
    // It allows the FSM to proceed through all 4 entry states regardless,  
    // hiding when the first error occurred.  
    reg correct_so_far;
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
// Password sequence (active-low)
// KEY_1 -> KEY_0 -> KEY_2 -> KEY_3
localparam PASS_1 = 4'b1101;
localparam PASS_2 = 4'b1110;
localparam PASS_3 = 4'b1011;
localparam PASS_4 = 4'b0111;

// Debouncer instance
wire debounced_press;
debounce uut_debounce (
    .clk(clk),
    .reset(reset),
    .buttons_in(keys_in),
    .debounced_press(debounced_press)
);

// State transition and flag logic
always @(posedge clk or posedge reset)
begin
    if (reset) begin
        current_state <= IDLE;
        correct_so_far <= 1'b1; // Assume correct at the start
    end else begin
        current_state <= next_state;
        // Update the correctness flag only when a button is pressed
        if (debounced_press) begin
            case (current_state)
                IDLE: begin
                    if (keys_in != PASS_1)
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
        correct_so_far <= 1'b0;

    end

    GOT_1: begin
        if (keys_in != PASS_2)
            correct_so_far <= 1'b0;
        end
    GOT_2: begin
        if (keys_in != PASS_3)
            correct_so_far <= 1'b0;
        end

        // No need for GOT_3 case here, final check is in combinational block
    endcase

end

// If we are moving to the IDLE state (e.g., via reset), reset the flag
if (next_state == IDLE) begin
    correct_so_far <= 1'b1;
end

end

end

// Next state and output logic
always @(*)
begin
    // Default assignments
    next_state = current_state;
    unlocked = 1'b0;
    failed = 1'b0;
    case (current_state)
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
IDLE: begin
    if (debounced_press)
        next_state = GOT_1;
    end
GOT_1: begin
    if (debounced_press)
        next_state = GOT_2;
    end
GOT_2: begin
    if (debounced_press)
        next_state = GOT_3;
    end
GOT_3: begin
    if (debounced_press) begin
        // This is the fourth and final press. Check the flag and the current input.
        if (correct_so_far && (keys_in == PASS_4))
            next_state = UNLOCKED;
        else
            next_state = FAILED;
        end
    end
end
UNLOCKED: begin
    unlocked = 1'b1;
    // Stay in this state until reset
end
FAILED: begin
    failed = 1'b1;
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
// Stay in this state until reset
end

default: begin

    next_state = IDLE;

end

endcase

end

endmodule
```

```
// Debounce module for multiple mechanical buttons
// It outputs a single pulse when any button is pressed and released.
module debounce(
    input clk,
    input reset,
    input [3:0] buttons_in,    // Changed to 4-bit input
    output reg debounced_press // Changed to a single pulse output
);
    // A 100MHz clock has a period of 10ns.
    // For a 20ms debounce time, we need 20ms / 10ns = 2,000,000 cycles.
    parameter COUNTER_MAX = 21'd1999999;
    reg [20:0] counter;
    reg [3:0] buttons_synced1, buttons_synced2;
    reg prev_buttons_state;
    wire any_button_pressed;

    // Synchronize the asynchronous button inputs to the system clock
    // to prevent metastability issues.

    always @(posedge clk or posedge reset) begin
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
if (reset) begin
    buttons_synced1 <= 4'b1111;
    buttons_synced2 <= 4'b1111;
end else begin
    buttons_synced1 <= buttons_in;
    buttons_synced2 <= buttons_synced1;
end
end

// An active-low button is pressed if its value is '0'
assign any_button_pressed = (buttons_synced2 != 4'b1111);
always @(posedge clk or posedge reset) begin
    if (reset) begin
        counter <= 0;
        prev_buttons_state <= 1'b0; // Assume not pressed initially
        debounced_press <= 1'b0;
    end else begin
        // We want to generate a single pulse when a button is first pressed.
        // We check if the previous state was unpressed and the current is pressed.
        if (prev_buttons_state == 1'b0 && any_button_pressed == 1'b1) begin
            // A button press has started, check if it's stable
            if (counter < COUNTER_MAX) begin
                counter <= counter + 1;
            end else begin
                // Debounce time has passed, button press is valid
                debounced_press <= 1'b1;
                prev_buttons_state <= 1'b1; // Update state to pressed
            end
        end
    end
end
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
end else if (any_button_pressed == 1'b0) begin
    // No buttons are pressed, reset everything
    counter <= 0;
    prev_buttons_state <= 1'b0;
    debounced_press <= 1'b0;
end else begin
    // Button is being held down, do nothing
    debounced_press <= 1'b0;
end
end
end
endmodule
```

// Drives a 4-digit common anode 7-segment display

```
module seven_segment_driver(
    input clk,
    input reset,
    input [3:0] char1, // Character for display 1 (left-most)
    input [3:0] char2,
    input [3:0] char3,
    input [3:0] char4, // Character for display 4 (right-most)
    output reg [6:0] segments, // {g,f,e,d,c,b,a}
    output reg [3:0] anodes // Anodes {disp4, disp3, disp2, disp1}
);
    // Clock divider to create a ~488Hz refresh rate for multiplexing
    // 100MHz / 204800 = 488.28 Hz
    reg [17:0] refresh_counter;
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

always @(posedge clk or posedge reset)

begin

if(reset)

refresh_counter <= 0;

else

refresh_counter <= refresh_counter + 1;

end

// The top 2 bits of the counter will cycle through 00, 01, 10, 11

// This gives each digit a ~122Hz refresh rate, which is plenty to avoid flicker

wire [1:0] digit_select = refresh_counter[17:16];

// Select which character to display based on the current active digit.

// The physical wiring of the anodes is different from the logical order,

// so we remap the characters here to match the physical display.

reg [3:0] char_to_display;

always @(*)

begin

case(digit_select)

// Physical digit 1 is activated when digit_select is 2'b10

2'b10: char_to_display = char1;

// Physical digit 2 is activated when digit_select is 2'b11

2'b11: char_to_display = char2;

// Physical digit 3 is activated when digit_select is 2'b00

2'b00: char_to_display = char3;

// Physical digit 4 is activated when digit_select is 2'b01

2'b01: char_to_display = char4;

default: char_to_display = 4'hF; // Blank

endcase



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

end

// Decoder for character codes to 7-segment patterns (active-low for common anode)

// Segments are arranged as {g,f,e,d,c,b,a}

always @(*)

begin

case(char_to_display)

4'h0: segments = 7'b1000001; // U

4'h1: segments = 7'b0101011; // n

4'h2: segments = 7'b1000111; // L

4'h3: segments = 7'b0100111; // c

4'h4: segments = 7'b0100011; // o

default: segments = 7'b1111111; // Blank

endcase

end

// Anode/Digit selection logic (active-low for common anode)

// This logic remains the same as it corresponds to the hardware multiplexing.

always @(*)

begin

case(digit_select)

2'b00: anodes = 4'b1110; // Activate physical display 3

2'b01: anodes = 4'b1101; // Activate physical display 4

2'b10: anodes = 4'b1011; // Activate physical display 1

2'b11: anodes = 4'b0111; // Activate physical display 2

default: anodes = 4'b1111; // All off

endcase

end

endmodule

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

OUTPUT RESULT:

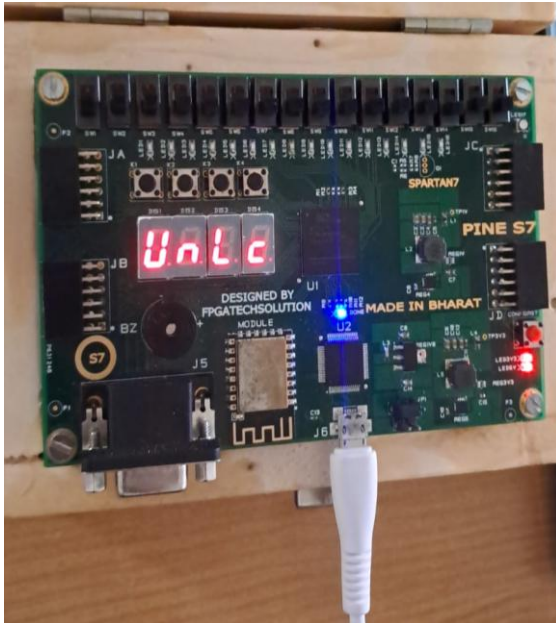


fig1: Successfully unlocked state

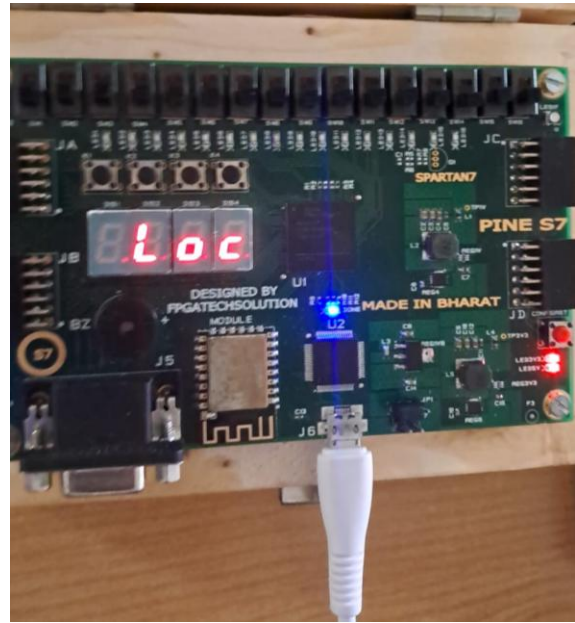


fig2: failed locked state

5.Conclusion and Future Work

5.1 Conclusion

The project has been effective in meeting the goal of creating and development of a powerful and sequential digital lock system with the help of Verilog HDL. The essentials of this were the deterministic and reliable verification of passwords through the core functionality which was a synchronized six-state Finite State Machine (FSM). Hardware input issues such as mechanical switch bouncing were successfully addressed by adding a specific counter-based debouncing hardware unit, which made sure that the physical key presses and the FSM transitions were only one-to-one related to each other. The modular design that included the FSM, debouncer and a multiplexed seven segment display driver was found to be efficient when used to utilize the hardware resources and this confirmed that the design was suitable to be implemented in a practical VLSI. This project confirms the need to consider realistic constraints (such as bounce) in purely sequential logic designs to come up with reliable embedded systems.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

5.2 Future Work

As additional security and user experience improvements to the digital lock, it is proposed to include the following future enhancements to the digital lock:

- 1) Implementation of Non-Volatile Memory: By including a system for monitoring external non-volatile memory (for example, Flash or EEPROM) for the reason of storing the access code. This allows the code to be user-programmable and survive a power-off event without going through a complete hardware reconfiguration.
- 2) Security preventive measure: Set up a time-out feature that, after a few consecutive incorrect entries, will immediately place the FSM in the FAILED state to avoid further attempts for a certain period of time (such as 30 seconds).
- 3) Acoustic Feedback: To offer auditory feedback (such as a beep when a key press, a cheerful tone upon unlocking, or a warning buzz when a failure), include a speaker driver and an audio tone generator.

5.3 Uses of Sequential Digital Lock Systems.

The basic design principles that were introduced in this project, such as the secure sequential access control, robust input handling, and clear visual feedback are extremely transferable and are used in various security and industrial areas:

- 1) Physical Access Control: This is employed in server racks, secure vaults, locking up industrial machines, and residential keyless entry systems whereby a reliable and multi-step authentication procedure is required.
- 2) Automotive Security: Inbuilt into vehicle immobilizers or hi-tech entry systems that demand the pressing of a series of buttons (such as a steering wheel code or keypad).
- 3) Industrial Control Systems: These are used in specialized equipment to ensure that the equipment cannot be accidentally or unauthorisedly operated, and therefore, a correct sequence of inputs is required to start up a high-power or dangerous system.
- 4) Consumer Electronics: They are used in electronic safes, security boxes and special vending machines that have a proprietary access code.